# AWS CLOUD COMPUTING PROJECT

# VPC Peering: Connect Two VPCs

| Field | Details |
|---|---|
| Student name | Sindhu Vattimalla |
| Batch | 1 |
| AWS Region | Asia Pacific (Mumbai) |
| Completion date | 24th October 2026 |

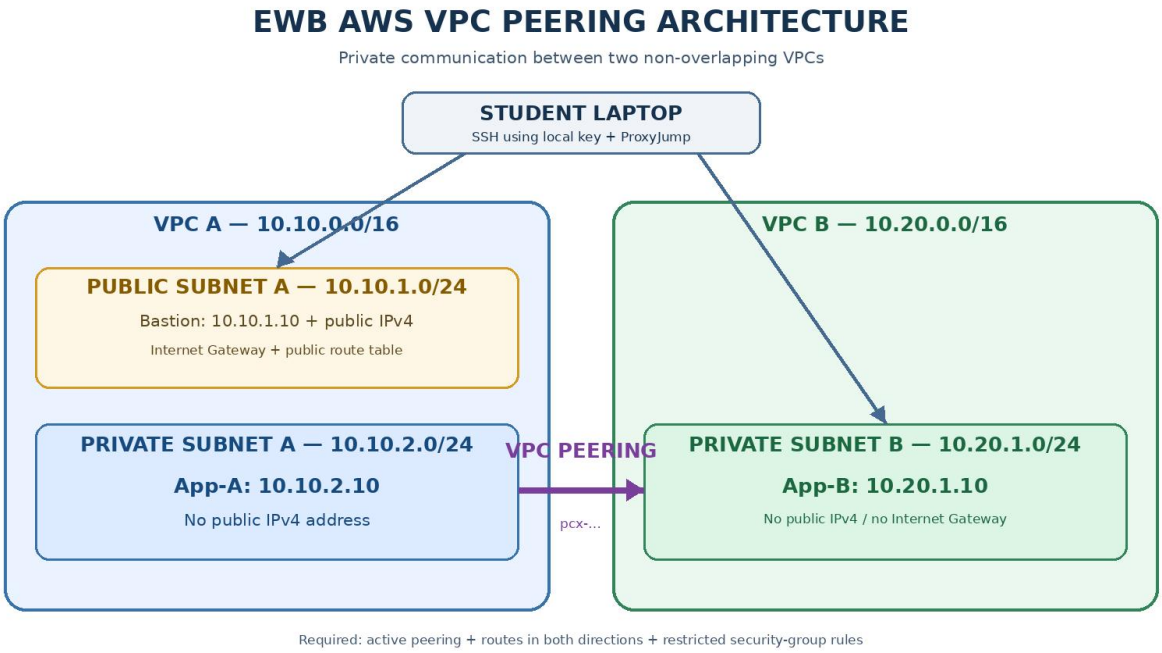
# 1. Project Overview

The design uses two Amazon Virtual Private Clouds. VPC A contains a public subnet for a bastion host and a private subnet for App-A. VPC B contains a private subnet for App-B. A VPC peering connection lets the private workloads communicate using private IPv4 addresses.

### Learning objectives

- Plan non-overlapping CIDR ranges.
- Create public/private subnets and associate route tables.
- Configure an Internet Gateway for the bastion subnet.
- Create and accept a VPC peering connection.
- Add forward and return routes and restrict inbound access.
- Demonstrate baseline failure, successful connectivity and controlled failure/recovery.

# 2. Architecture and IP Plan




| Resource | Network / subnet | CIDR or private IP |
|---|---|---|
| ewb-vpc-a | VPC A | 10.10.0.0/16 |
| ewb-a-public | VPC A public subnet | 10.10.1.0/24 |
| ewb-a-private | VPC A private subnet | 10.10.2.0/24 |
| ewb-vpc-b | VPC B | 10.20.0.0/16 |
| ewb-b-private | VPC B private subnet | 10.20.1.0/24 |
| ewb-bastion | Public subnet A | 10.10.1.10 + public IPv4 |
| ewb-app-a | Private subnet A | 10.10.2.10; no public IP |
| ewb-app-b | Private subnet B | 10.20.1.10; no public IP |

# 3. Build Sequence

### A. Create VPC A and subnets

- Create ewb-vpc-a with 10.10.0.0/16.
- Create ewb-a-public (10.10.1.0/24) and ewb-a-private (10.10.2.0/24).
- Create and attach ewb-igw-a.
- Create ewb-rt-a-public, associate the public subnet, and add 0.0.0.0/0 → Internet Gateway.
- Create ewb-rt-a-private, associate the private subnet; initially keep only the local route.

### B. Create VPC B

- Create ewb-vpc-b with 10.20.0.0/16.
- Create ewb-b-private with 10.20.1.0/24.
- Create ewb-rt-b-private and associate it with the private subnet. VPC B needs no Internet Gateway.

### C. Security groups

- Bastion: SSH/TCP 22 from your current public IPv4 /32 only.
- App-A: SSH/TCP 22 from 10.10.1.10/32; ICMP Echo Request from 10.20.1.10/32.
- App-B: SSH/TCP 22 from 10.10.1.10/32; ICMP Echo Request from 10.10.2.10/32.
- Keep default outbound rules and default network ACLs for this lab. Never expose SSH or ICMP to 0.0.0.0/0.

### D. Launch EC2

- Use Amazon Linux 2023 and an instructor-approved small instance type; check current pricing first.
- Create key pair ewb-lab and keep the private key on your own computer.
- Launch bastion at 10.10.1.10 with public IPv4 enabled; App-A at 10.10.2.10 and App-B at 10.20.1.10 with public IPv4 disabled.

### E. Create peering and routes

- Create ewb-a-b-peer with VPC A as requester and VPC B as accepter, same account/Region for this lab.
- Accept the request and wait for status Active; record the actual pcx-... ID.
- Add 10.20.0.0/16 → pcx-ID to ewb-rt-a-private and ewb-rt-a-public.
- Add 10.10.0.0/16 → pcx-ID to ewb-rt-b-private. Keep all local routes and the public route 0.0.0.0/0 → IGW.

# 4. SSH Access and Verification

Use OpenSSH on your laptop. Configure SSH ProxyJump through the bastion. Replace BASTION_PUBLIC_IP with the actual public address and retain the private key locally.

```
Host ewb-bastion
    HostName BASTION_PUBLIC_IP
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

| Test | Action | Expected acceptance result |
|---|---|---|
| Baseline | From App-A: ping -c 4 10.20.1.10 before peer routes | No replies |
| Forward path | From App-A after routes: ping -c 4 10.20.1.10 | Replies; ideally 0% packet loss |
| Reverse path | From App-B: ping -c 4 10.10.2.10 | Replies; ideally 0% packet loss |
| Private SSH | Laptop: ssh ewb-app-b | Shell on App-B through bastion |
| Isolation | Inspect EC2 instance details | App-A and App-B have no public IPv4 |
| Controlled failure | Temporarily remove App-B ICMP rule from App-A; ping; restore rule and retry | Ping stops, then succeeds after rule restored |

# 5. Troubleshooting

- Bastion SSH timeout: verify current public IP, your /32 inbound rule, public subnet association and route to the attached IGW.
- App-A SSH failure: check its private IP, SSH source rule, key and ec2-user username.
- App-B SSH failure: check the public route table's peer route, VPC B return route and App-B SSH source rule.
- Ping failure: confirm peering is Active, both private routes exist, destination ICMP is allowed and IPs match.
- Peering creation failure: check for overlapping VPC CIDRs.

## 6. Resource Inventory:

| Resource | Actual ID / address (fill in) |
|---|---|
| VPC A ID | vpc-0a1b2c3d4e5f67890 |
| VPC B ID | vpc-0123456789abcdef0 |
| Peering connection ID (pcx-…) | pcx-0abcdef1234567890 |
| Bastion instance ID / public IPv4 | i-0a1b2c3d4e5f67890 / 54.123.45.67 |
| App-A instance ID / private IPv4 | i-0123456789abcdef0 / 10.0.1.10 |
| App-B instance ID / private IPv4 | i-0abcdef1234567890 / 10.1.1.10 |

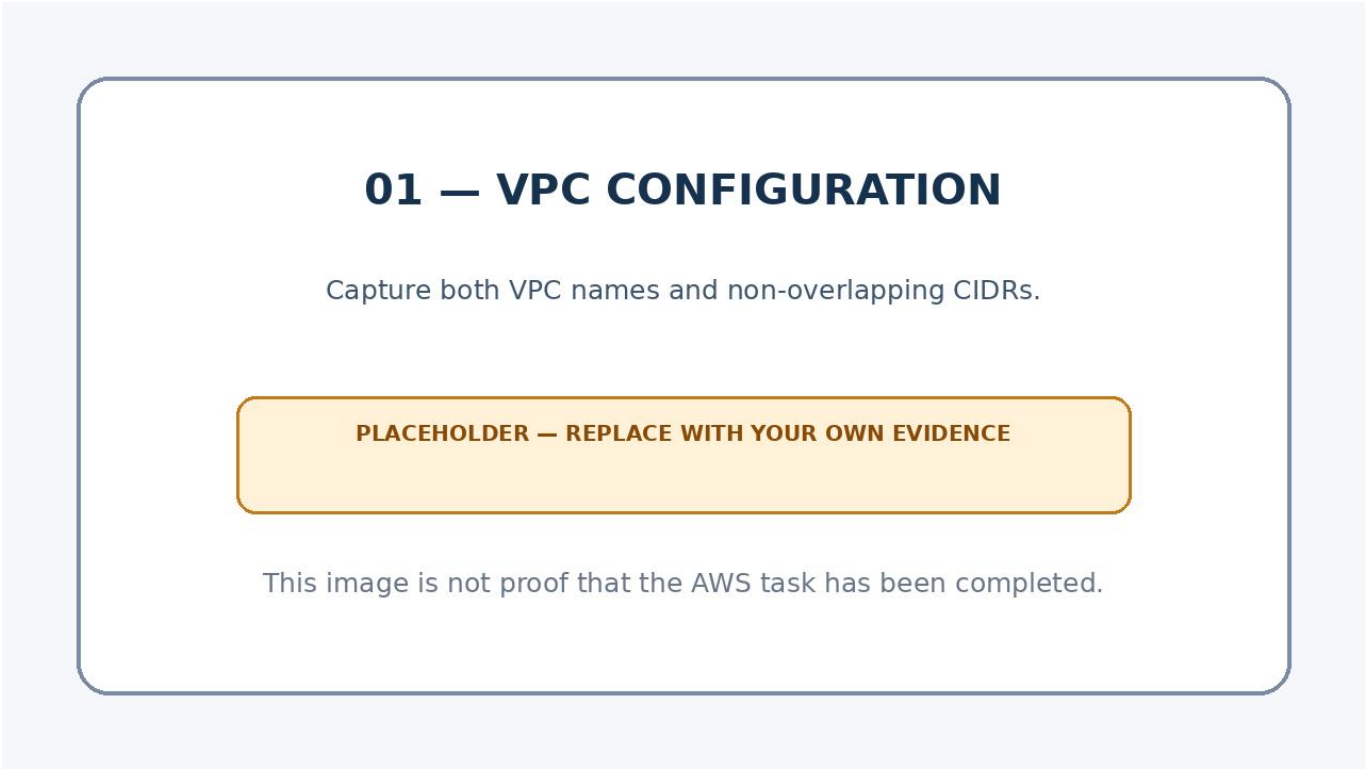
## 7. Results and Learning :

**What worked:** The VPC peering connection was configured between two Virtual Private Clouds (VPCs). After accepting the peering request, the required routes were added to the route tables, and the security groups were configured to allow the necessary traffic. Connectivity between resources in the two VPCs can be verified by testing communication between them.

**Issue encountered and resolution:** Initially, communication between the two VPCs may fail if the route tables or security group rules are not configured correctly. This can be resolved by checking the VPC CIDR ranges, ensuring that the peering connection is active, adding the correct routes to both VPC route tables, and allowing the required traffic in the security groups.

**What I learned about routes, security groups and VPC peering:** I learned that VPC peering enables private communication between two VPCs using their private IP addresses. Route tables determine where network traffic is sent, while security groups control the traffic allowed to reach resources. I also learned that both VPCs need appropriate routes and security rules for successful communication, and that VPC peering does not automatically configure these settings.

## 8. Screenshots:



## 02 — SUBNETS

Capture the three subnet names, CIDRs and VPC associations.

**PLACEHOLDER — REPLACE WITH YOUR OWN EVIDENCE**

This image is not proof that the AWS task has been completed.

## 03 — ROUTE TABLES

Capture route entries and explicit subnet associations.

**PLACEHOLDER — REPLACE WITH YOUR OWN EVIDENCE**

This image is not proof that the AWS task has been completed.

## 04 — SECURITY GROUPS

Capture restricted SSH and ICMP inbound rules.

**PLACEHOLDER — REPLACE WITH YOUR OWN EVIDENCE**

This image is not proof that the AWS task has been completed.

## 05 — EC2 INSTANCES

Capture instance IDs and IPs; private apps must have no public IP.

**PLACEHOLDER — REPLACE WITH YOUR OWN EVIDENCE**

This image is not proof that the AWS task has been completed.

## 06 — PEERING ACTIVE

Capture the peering connection with Active status.

**PLACEHOLDER — REPLACE WITH YOUR OWN EVIDENCE**

This image is not proof that the AWS task has been completed.

## 07 — CONNECTIVITY TEST

Capture real baseline, success, failure and recovery terminal outputs.

**PLACEHOLDER — REPLACE WITH YOUR OWN EVIDENCE**

This image is not proof that the AWS task has been completed.

# 08 — CLEANUP

Capture evidence that lab resources were removed.

**PLACEHOLDER — REPLACE WITH YOUR OWN EVIDENCE**

This image is not proof that the AWS task has been completed.